

Prompt 2

KKC TEXT ADVENTURE — PROMPT 2 OF 180

University Vertical Slice: Richer Locations, NPCs, Economy, and Time

This prompt builds directly on the working scaffold from Prompt 1.
Do not restructure, rename, or refactor anything from Prompt 1.
Only add to what exists.

HARD BANS — UNCHANGED FROM PROMPT 1

No external APIs. No HTTP clients. No LLM integration. No Anthropic, OpenAI, or any remote service. No web server. No frontend.
The game must remain fully offline.

No new abstractions unless explicitly required by this prompt.
No new files unless listed in this prompt.
Do not refactor working Prompt 1 code.

CANON RESTRAINT RULE — UNCHANGED

All new location descriptions and NPC content must be grounded in the published books. If a physical detail is not confirmed in canon, keep it modest and generic. Do not invent architecture, layout, or social detail with confidence.

Apply this rule to every piece of authored text in this prompt.

GOAL OF THIS PROMPT

By the end of Prompt 2, the player should be able to:

- Wake in their room and decide whether to spend money on breakfast or save it for tuition
- Move through 11 navigable University locations with distinct atmosphere and accurate exits
- Encounter Simmon and Wilem as present, reactive NPCs with correct voice and location
- Work at Anker's to earn drabs
- See money pressure as real: lodging costs, meals cost, funds deplete

- Watch time advance across a full in-game day with consequence
- Type `talk to [name]` and receive a character-true response
- See hunger and fatigue rise meaningfully if ignored

SCOPE OF THIS PROMPT

This prompt covers:

1. Expand seed data to 11 canon-grounded University locations
2. Add 3 NPCs: Simmon, Wilem, Anker
3. Add economy actions: eat, sleep/rest, busk
4. Add the `talk to [name]` command
5. Wire hunger and fatigue into time advancement
6. Add a tuition deadline tracker to world_state
7. Expand locationFlavour.ts for all 11 locations
8. Add tests for economy and NPC presence

This prompt does NOT cover:

- Sympathy system (Prompt 3)
- Ambrose conflict (Prompt 4)
- Eolian / Imre (Prompt 5)
- Academic rank advancement (Prompt 6)
- LLM narration (much later)

NEW FILES IN THIS PROMPT

Add exactly these files. Do not add others.

```
src/  
  engine/  
    economy.ts    ← eat, sleep, busk logic  
    npcEngine.ts  ← talk command, NPC presence checks  
  content/  
    npcProfiles.ts ← authored NPC character data  
    rumourPool.ts ← per-location rumour strings (stub, 2 per location)  
  tests/  
    economy.test.ts  
    npcEngine.test.ts
```

Modify these existing files:

```
src/db/seed.ts      ← expand to 11 locations + 3 NPCs  
src/engine/actions.ts ← add eat, sleep, busk, talk commands  
src/engine/time.ts  ← wire hunger/fatigue into advanceTime  
src/content/locationFlavour.ts ← expand to all 11 locations  
src/types/index.ts  ← add TuitionState, NPCProfile interfaces
```

TYPES — additions to src/types/index.ts

Add these interfaces. Do not remove or change existing ones.

```
export interface TuitionState {
  amount_drabs: number;    // what is owed this term
  due_on_day: number;      // day_number when it must be paid
  paid: boolean;
  overdue: boolean;
}
```

```
export interface NPCProfile {
  id: string;
  name: string;
  era: string;
  home_location_id: string; // where they are found by default
  temperament: string;     // brief: e.g. 'warm, perceptive, decent'
  speech_style: string;     // brief: e.g. 'direct, uses few words'
  known_topics: string[];   // topics this NPC will discuss
  taboo_topics: string[];   // topics this NPC will not discuss
  greeting_pool: string[];  // 3 greetings, chosen deterministically
  exit_lines: string[];     // 2 ways this NPC ends a conversation
}
```

Also add `tuition_state` to `PlayerState`:

```
tuition_state: TuitionState;
```

Update `initDefaultPlayerState()` in `engine/state.ts` to include:

```
tuition_state: {
  amount_drabs: 30, // 3 jots — canon-appropriate for early E'lir
  due_on_day: 14,   // two weeks into term
  paid: false,
  overdue: false,
}
```

SEED EXPANSION — src/db/seed.ts

Expand `runSeed(db)` to insert 11 locations total (INSERT OR IGNORE).
Keep the 5 from Prompt 1 unchanged. Add these 6:

```
university_mews_room
  name: "Kvothe's room, the Mews"
  exits:
```

out -> university_mews_corridor
tier: 1, cluster_id: 'university', era: 'university'
Note: Small, spartan, cheap. This is where the player starts each morning. Canon establishes Kvothe lives in the Mews in early terms.

university_mews_corridor
name: "the Mews corridor"
exits:
in -> university_mews_room
south -> university_courtyard
tier: 2, cluster_id: 'university', era: 'university'
Note: Tier 2 — implied geography. Keep description generic.

university_courtyard
name: "the University courtyard"
exits:
north -> university_mews_corridor
east -> university_mains
west -> university_ankers
tier: 1, cluster_id: 'university', era: 'university'
Note: Central open space. Canon references courtyard areas.

university_ankers
name: "Anker's inn"
exits:
east -> university_courtyard
tier: 1, cluster_id: 'university', era: 'university'
Note: Canon-confirmed student social space. Kvothe works and performs here. Keep description modest — exact layout not detailed in text.

university_fishery_outer
name: "the Fishery, outer workspace"
exits:
south -> university_mains
tier: 1, cluster_id: 'university', era: 'university'
Note: The Artificery's working floor. Kilvin's domain. Canon describes heat, metal work, students at benches. Keep it sensory but do not invent specific room structure.

university_mains_hall
name: "the Mains, lecture hall"
exits:
out -> university_mains
tier: 1, cluster_id: 'university', era: 'university'
Note: Where classes are held. Separate from the Mains exterior.

Update exits on existing locations to connect the new ones:

university_mains: add exit west -> university_fishery_outer
add exit west -> university_courtyard
(remove duplicate west if needed — choose
one: courtyard is the better connection)

Actually: university_mains exits should be:

north -> university_artificery
east -> university_archives_exterior
south -> university_medica
west -> university_courtyard

Also insert 3 NPCs:

simmon

name: "Simmon"
location_id: university_ankers (his default daytime location)
era: university
temperament: "warm, perceptive, genuinely decent, not naive"
speech_style: "easy and direct, quick to laugh, notices things
others miss, does not pry but does not ignore
distress either"

wilem

name: "Wilem"
location_id: university_mains (his default location near classes)
era: university
temperament: "reserved, loyal, sceptical, economical with words"
speech_style: "brief, dry, Siaru accent shapes his phrasing,
rarely volunteers information, trust runs deep
once given"

anker

name: "Anker"
location_id: university_ankers
era: university
temperament: "practical, tolerant, neither warm nor cold"
speech_style: "functional, inn-keeper terse, fair but not
generous"

NPC PROFILES — src/content/npcProfiles.ts

Export NPC_PROFILES: Record<string, NPCProfile>

Write profiles for: simmon, wilem, anker

Rules for all profiles:

- greeting_pool: exactly 3 entries, chosen deterministically by state.day_number % 3
- exit_lines: exactly 2 entries
- All dialogue is original prose — do NOT reproduce Rothfuss lines
- Voice must match the character as established in the books
- known_topics and taboo_topics use plain English strings

Simmon profile notes:

- known_topics: classes, tuition, students, music, Wilem, University gossip, how Kvothe is doing
- taboo_topics: his family's money, his noble background
- Greetings should feel warm and unpretentious
- He notices if the player looks tired or troubled, but doesn't force the issue

Wilem profile notes:

- known_topics: classes, Cealdir customs, money, Archives, Kilvin's Fishery work
- taboo_topics: personal family matters, Siaru home life
- Greetings should be brief and real — not cold, but measured
- His Siaru background is part of his character; do not flatten it

Anker profile notes:

- known_topics: rooms, meals, work shifts, local gossip (surface level only), what's available at the inn
- taboo_topics: students' private business
- Greetings are transactional and not unfriendly

NPC ENGINE — src/engine/npcEngine.ts

Implement:

getNPCProfile(npc_id: string): NPCProfile | null

— Looks up NPC_PROFILES by id. Returns null if not found.

isNPCPresent(npc: NPC, state: PlayerState): boolean

— For MVP: return true if npc.location_id === state.location_id
This is the simple version. NPC schedules come in a later prompt.

talkToNPC(
 npc_id: string,
 topic: string | null,
 state: PlayerState,
 db

): string

— Finds the NPC in the current location.

- If NPC not present: return an in-world "not here" line, e.g. "Simmon isn't here right now."
- If NPC present but topic is on their taboo_topics list: return a brief deflection in their voice. Do not explain the mechanic. Stay in-world.
- If NPC present and topic is known or null: return their greeting (deterministic from day_number) followed by a brief in-character response to the topic. For null topic (just "talk to simmon"), return greeting only.
- All responses are deterministic local text from npcProfiles.ts. No LLM. No randomness.
- Response length: 2-4 sentences maximum. Do not monologue.

parseNPCCommand(input: string): { npc_id: string; topic: string | null } | null

- Parses inputs like:
 - "talk to simmon" -> { npc_id: 'simmon', topic: null }
 - "talk to wilem about work" -> { npc_id: 'wilem', topic: 'work' }
 - "speak to anker" -> { npc_id: 'anker', topic: null }
 - "ask simmon about tuition" -> { npc_id: 'simmon', topic: 'tuition' }
- Returns null if pattern does not match.
- Recognise: "talk to", "speak to", "ask [name] about"
- NPC name matching is case-insensitive.
- Topic is everything after "about" if present, otherwise null.

ECONOMY — src/engine/economy.ts

Implement these pure functions. All return a new PlayerState.
None write to the DB — the caller in actions.ts handles persistence.

- eat(state: PlayerState): { newState: PlayerState; message: string }
- Cost: 3 drabs (a basic meal, canon-consistent for student budgets)
 - If player has ≥ 3 drabs:
 - deduct 3 drabs
 - reduce hunger by 40 (min 0)
 - return success message: brief, in-world, not cheerful-generic e.g. "You eat something plain and filling."
 - If player has < 3 drabs:
 - unchanged state
 - message: "You haven't the coin for even a modest meal."

- sleep(state: PlayerState): { newState: PlayerState; message: string }
- Sleeping at Anker's costs 2 jots (20 drabs) per night.
 - Player must be at university_ankers or university_mews_room to sleep (check state.location_id).
 - If wrong location:
 - return message: "You need somewhere to actually sleep."

state unchanged.

- If at a valid sleep location and has ≥ 20 drabs:
 - deduct 20 drabs (if at Anker's — Mews room already paid for term, no deduction there for now; treat Mews room as free)
 - set fatigue to 0
 - reduce hunger by 20 (min 0)
 - advance time by 2 steps (night -> morning)
 - return message: a brief, original in-world line about waking
- If at Anker's but not enough drabs:
 - state unchanged
 - message: "You can't afford the room tonight."
- Note: Mews room is pre-paid for the term in early University era. Sleeping there is free. Anker's costs. This is canon-consistent.

busk(state: PlayerState): { newState: PlayerState; message: string }

- Player must be at university_ankers.
- If wrong location:
 - return message: "There's nowhere to play for coin here."
 - state unchanged.
- If at Anker's:
 - Calculate earnings deterministically:
 - base = 5 drabs
 - if eolian_standing ≥ 40 : add 3 drabs
 - if university_social ≥ 50 : add 2 drabs
 - if fatigue ≥ 60 : subtract 2 drabs (min 1)
 - if hunger ≥ 70 : subtract 1 drab (min 1)
 - Add earnings to money_drabs.
 - Increase eolian_standing by 1 (max 100).
 - Advance time by 1 step (busking takes an evening).
 - Return an in-world message that mentions the earnings without being chirpy. Do not use the word "earned". Something like:
 - "You play for two hours. People leave coins on the edge of the bar. [X] drabs in all."
 - Vary the message using day_number % 3 from a pool of 3 lines.

checkTuitionDeadline(state: PlayerState): PlayerState

- Called at the end of each turn in actions.ts after state changes.
- If day_number $\geq$ tuition_state.due_on_day and tuition_state.paid === false and tuition_state.overdue === false:
 - set tuition_state.overdue = true
 - return updated state
- Otherwise return state unchanged.
- The overdue flag affects narration in later prompts. For now, just set it accurately.

payTuition(state: PlayerState): { newState: PlayerState; message: string }

- If money_drabs $\geq$ tuition_state.amount_drabs and not already paid:

deduct amount
set tuition_state.paid = true
message: "You pay your tuition. The weight of it leaves you."
— If already paid:
message: "Your tuition is already settled."
— If insufficient funds:
message: "You don't have enough."
— State unchanged on failure.

TIME ADVANCEMENT — update src/engine/time.ts

Modify advanceTime() to also update hunger and fatigue:

- Each time step increases hunger by 8 (max 100)
- Each time step increases fatigue by 6 (max 100)
- Sleep resets these (handled in economy.ts — do not duplicate)
- Eating reduces hunger (handled in economy.ts — do not duplicate)
- advanceTime still returns a new PlayerState without mutating input

ACTIONS — update src/engine/actions.ts

Add these commands to dispatch():

eat

- > call eat(state) from economy.ts
- > call checkTuitionDeadline(newState)
- > return message + status line if hunger changed significantly

sleep / rest

- > call sleep(state)
- > on success: load location and npcs, compute accessibleExits, return sleep message + renderLocation()
- > on failure: return failure message

busk / play / perform

- > call busk(state)
- > return busk message

pay tuition / pay fees

- > call payTuition(state)
- > return result message

talk to [name] / speak to [name] / ask [name] about [topic]

- > call parseNPCCommand(input)
- > if null: return narrator.renderFallback()

-> if parsed: call talkToNPC(), return result

After every command that changes state, call checkTuitionDeadline()
on the returned newState before passing it back to repl.ts.

If tuition_state.overdue becomes newly true this turn, append a
brief in-world warning line to the output:

"A thought nags at you. Your tuition was due."

RUMOUR POOL — src/content/rumourPool.ts

Export RUMOUR_POOL: Record<string, string[]>

Each location_id maps to an array of exactly 2 rumour strings.
These are things overheard or noticed — not quest hooks, not lore
dumps. Brief, atmospheric, in-world. Apply canon restraint rule.

Do not resolve any series mystery. Do not invent confident lore.
Rumours can be mundane: student gossip, weather, tuition complaints,
overheard fragments.

Provide entries for all 11 locations.

For now these are unused — they will be wired into NPC conversation
and idle observation in Prompt 3. Include the file and data only.

LOCATION FLAVOUR — update src/content/locationFlavour.ts

Add exactly 4 flavour lines for each of the 6 new locations:

university_mews_room
university_mews_corridor
university_courtyard
university_ankers
university_fishery_outer
university_mains_hall

Keep existing entries for the original 5 locations unchanged.
Original prose only. Canon restraint rule applies.

TESTS — tests/economy.test.ts

Import eat, sleep, busk, payTuition, checkTuitionDeadline from

src/engine/economy.ts.

Import initDefaultPlayerState from src/engine/state.ts.

Use initDefaultPlayerState() as the base state for all tests.

Tests:

1. eat() with sufficient funds deducts 3 drabs and reduces hunger
2. eat() with 0 drabs returns failure message and unchanged state
3. sleep() at university_mews_room sets fatigue to 0 and does not deduct drabs
4. sleep() at university_mains returns failure message
5. busk() at university_ankers adds drabs to money_drabs
6. busk() at university_mains returns failure message
7. payTuition() with sufficient funds sets tuition_state.paid = true
8. payTuition() with insufficient funds returns failure message
9. checkTuitionDeadline() sets overdue when day_number >= due_on_day and paid is false
10. checkTuitionDeadline() does not set overdue when already paid

TESTS — tests/npcEngine.test.ts

Import parseNPCCCommand, isNPCPresent, talkToNPC from src/engine/npcEngine.ts.

Import initDefaultPlayerState from src/engine/state.ts.

Use an in-memory SQLite database (new Database(':memory:')) with runMigrations() and runSeed() called before tests.

Tests:

1. parseNPCCCommand("talk to simmon") returns { npc_id: 'simmon', topic: null }
2. parseNPCCCommand("ask wilem about classes") returns { npc_id: 'wilem', topic: 'classes' }
3. parseNPCCCommand("look around") returns null
4. isNPCPresent() returns true when npc.location_id matches state.location_id
5. isNPCPresent() returns false when locations differ
6. talkToNPC('simmon', null, state, db) when player is at university_ankers returns a non-empty string
7. talkToNPC('simmon', null, state, db) when player is at university_mains returns a "not here" message

ARCHITECTURE REMINDERS

These rules from Prompt 1 still apply:

- narration/* never reads from or writes to the DB
- narration/* never mutates PlayerState
- engine/* owns all state changes
- repl.ts saves state exactly once per turn
- No `any` types
- No external API calls
- No unused imports or files

economy.ts and npcEngine.ts are engine-layer files.
They may read from the DB (npcEngine) or not (economy — pure).
They must not call narrator methods directly.

SUCCESS CRITERIA

This prompt is complete when:

- `look` renders one of 11 distinct locations
- `go north/south/etc` navigates the full 11-location map
- `eat` costs 3 drabs and reduces hunger
- `sleep` at Mews room sets fatigue to 0, costs nothing
- `busk` at Anker's earns drabs deterministically
- `pay tuition` works when funds are sufficient
- `talk to simmon` returns a character-true greeting
- `talk to wilem` when Wilem is not present returns "not here"
- `ask anker about work` returns a brief in-character line
- Hunger rises across a full day without eating
- Fatigue rises across a full day without sleeping
- Tuition overdue flag is set correctly when deadline passes
- All 17 new tests pass alongside the 9 from Prompt 1
- No network calls at any point
- No refactoring of Prompt 1 code

=====
END OF PROMPT 2
=====